ROLL NO: 240701622 **NAME: Krishna Kumar MS**

1.Comparative study of low-code development versus traditional application development approaches.

1. Aim

To conduct a comparative study between low-code development and traditional application development approaches, analyzing their stages, workflows, and technical differences.

2. Introduction to Traditional Approach

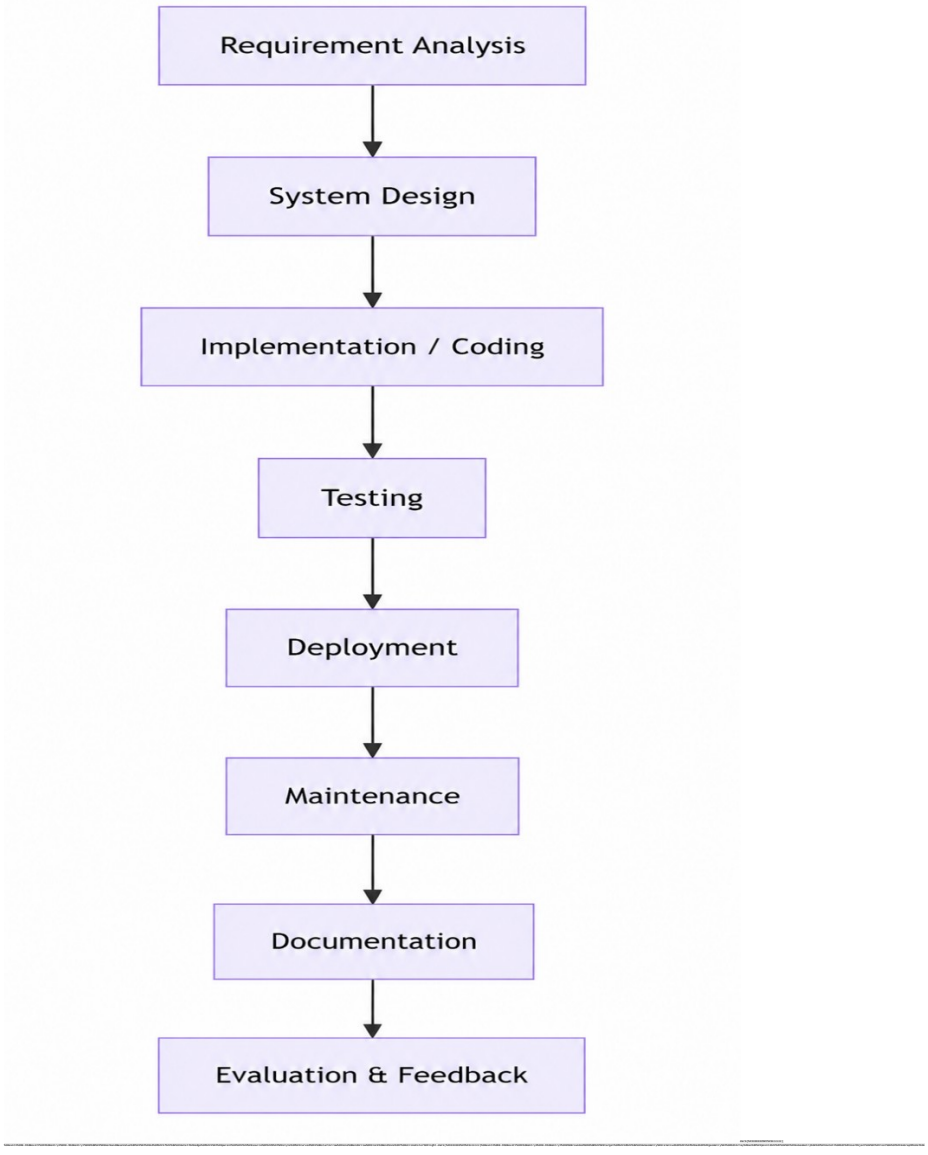
The traditional approach to software development relies on manual, line-by-line programming using standard text editors and integrated development environments (IDEs). Developers must manually manage database schemas, API integrations, and server deployments. This highly skill-dependent process requires deep technical expertise in languages such as Java, Python, or C++. While this method offers ultimate flexibility and full control over the codebase and architecture, it typically involves longer development cycles, extensive budgeting, and meticulous bug-testing

3. Introduction to Low-Code Approach

Low-Code Application Platforms (LCAPs) replace intensive hand-coding with cloud-based, visual development environments. They utilize drag-and-drop elements, pre-built visual models, and automated deployment pipelines. This approach flattens the learning curve, allowing "citizen developers" with minimal or no coding experience to build functional applications rapidly. By automating repetitive programming tasks, low-code drastically speeds up time-to-market and lowers initial development costs, though it may introduce limitations in deep customization and platform dependency.

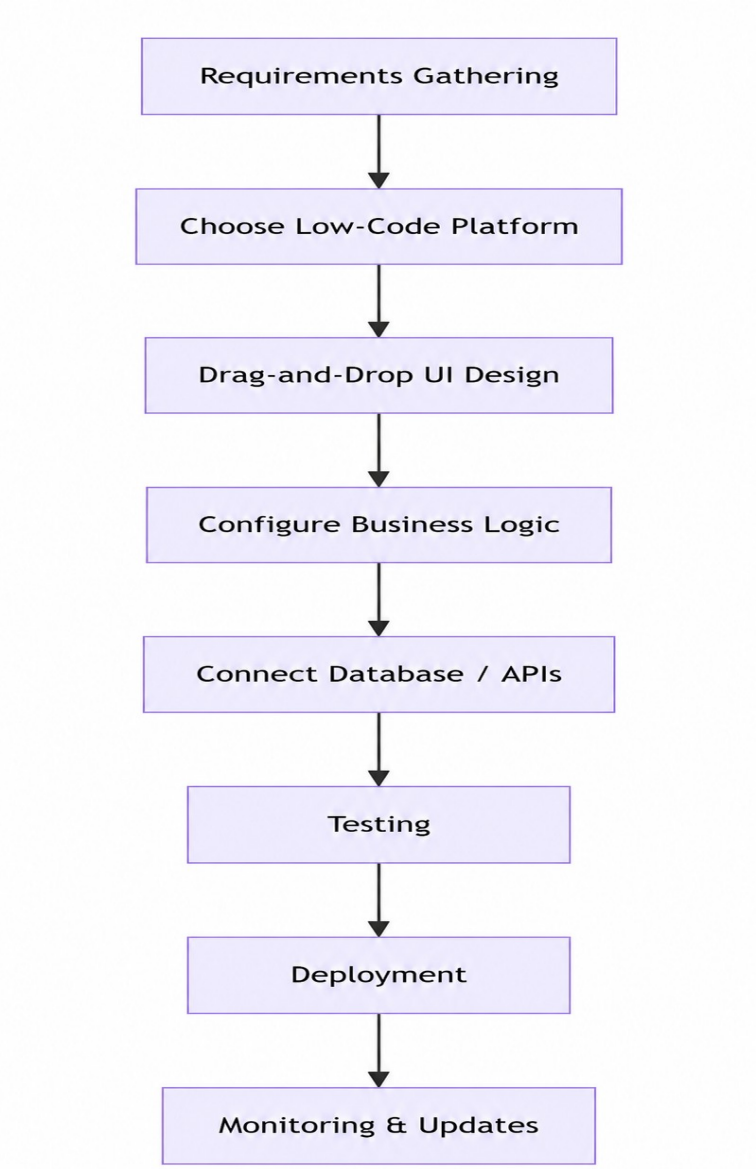
4. Diagram for Stages in Traditional Approach

The Traditional Software Development Life Cycle (SDLC) is a structured process where applications are developed through sequential stages. Each stage is completed before moving to the next, ensuring quality and proper documentation.



5. Diagram for Stages in Low-Code Development

Low-code development uses visual interfaces, drag-and-drop components, and minimal coding to build applications quickly. It reduces development time while simplifying deployment and maintenance.



6. Comparative Analysis (Tabulation)

Feature	Traditional Development	Low-Code Development
Development Speed	Slow; requires hand-coding everything from scratch.	Fast; reduces time by up to 50% to 70% using pre-built modules.
Skill Required	High; requires skilled software engineers/developers.	Low; enables "citizen development" with drag-and-drop tools.
Customization	Infinite; developer has total control over every line of code.	Limited; restricted to the capabilities and components offered by the platform.
Cost	Higher initial setup, developer salaries, and long-term maintenance.	Lower upfront costs, but involves ongoing platform licensing or subscription fees.
Maintenance & Updates	Laborious; each bug fix and update must be manually coded and tested.	Simplified; updates are centrally managed and deployed by the platform provider.
Vendor Lock-in	None; the business has 100% ownership of the source code.	High; the application depends on the specific vendor's infrastructure and ecosystem.
Best Use Cases	Complex, high-performance, or highly secured enterprise-level systems.	Prototyping, workflow automation, and internal business tools.

7. Result

The comparative study successfully demonstrates that while **traditional development** offers maximum flexibility, granular control, and custom scalability for mission-critical apps, **low-code development** provides a highly agile, cost-effective, and rapid solution for building, deploying, and maintaining business applications. Organizations often benefit from a hybrid approach, leveraging low-code for quick front-end and workflow tasks while keeping traditional development for complex, custom back-end needs.